VaultX: MongoDB and PyMongo Based Document Metadata Analytics

Course project report

Implementation: React/Vite, Express/Mongoose, MongoDB, PyMongo, scikit-learn

Evaluation status: Reproducible synthetic demo; results from a live vault database must be run separately.

1. Introduction

VaultX is a digital document management prototype. The application stores user accounts, document metadata, folders, and activity logs in MongoDB; binary files are uploaded through Cloudinary. The MongoDB course extension adds a Python analytics workflow using PyMongo to create indexes, seed isolated demo records, run aggregation queries, train two machine-learning models, and preserve evaluation results. This makes MongoDB both the operational data store and a source for reproducible analysis.

2. Project objective

1. Manage document metadata in MongoDB and support folder navigation, file search, and user activity history.
2. Use **PyMongo** to query the existing collections, create useful indexes, aggregate file statistics, and write model-run records.
3. Predict a document's size in MB with a regression model and classify whether it is at least 5 MB with a classification model.
4. Measure regression with **R^2** and **MAE**; measure classification with **accuracy, precision, recall, and F1**.
5. Keep demonstration records separate from real vault records and avoid processing file contents or sensitive account fields.

3. Dataset description

The operational `files` collection is created by the existing Express/Mongoose backend. Each file document includes `user` (owner ObjectId), `fileName`, `fileType` (MIME type), `fileSize` (legacy display string), `sizeBytes` (new exact numeric size), `isFolder`, `parentId`, and `createdAt`. Folder records have no file size and are excluded from training. The Python projection reads only `fileName`, `fileType`, `fileSize`, `sizeBytes`, and `isFolder`.

For a reproducible course demonstration, `demo_documents()` creates 600 synthetic metadata records with six MIME types: PDF, DOCX, XLSX, JPEG, PNG, and MP3. The random seed is 42. File sizes are sampled from type-specific Gaussian distributions and clipped to a positive minimum. These records can be upserted into a separate `ml_demo_files` collection using `seed-demo`; no synthetic record is put into the application's `files` collection. The generated-only evaluation uses the same deterministic records without MongoDB and is explicitly a **demo result**, not a claim about actual users' documents.

4. MongoDB and PyMongo implementation

The application continues to use Mongoose for transactions such as upload and folder creation. The analytics module (`analytics/vault_ml.py`) uses `MongoClient` from PyMongo, validates the connection with `ping`, and reads the `files` or `ml_demo_files` collection. New uploads store `sizeBytes` as a number while retaining `fileSize` for current UI compatibility.

`init-db` creates a compound `files` index on `(user, parentId, createdAt)` for owner and folder views, an `activities` index on `(user, createdAt)` for recent logs, a unique `demoId` index for repeatable upserts, and a descending `createdAt` index for model-run history. `summary` executes a MongoDB aggregation pipeline: `$match` non-folder documents with numeric sizes, `$group` by MIME type to count files and sum/average bytes, and `$sort` by count. MongoDB-backed training writes metrics and source information to `ml_model_runs`. Existing app searches use a case-insensitive regular expression; that search pattern is not fully optimized by the added index.

The database URI is supplied through `MONGO_URI` or `--mongo-uri`, and `--database` can select a database explicitly. The command does not print credentials. The analytics code does not request Cloudinary URLs, passwords, billing data, or file contents.

5. Data preprocessing

Folders, missing sizes, zero sizes, and invalid legacy size strings are excluded. Numeric `sizeBytes` is preferred and converted from bytes to MB. Older strings such as `"2.35 MB"` are parsed as a fallback, which is less precise because those values were rounded at upload time. MIME type is normalized to lowercase. File extension is extracted from the filename; the feature set is **MIME type, extension, and filename length**. Categorical fields are one-hot encoded by `DictVectorizer` inside each model pipeline. File size is never included in the input features, so the large-file label cannot leak directly into the classifier.

The target for regression is size in MB. The binary target for classification is `1` when size is at least 5 MB and `0` otherwise. The data is divided into a 75% training set and a 25% held-out test set with random seed 42; the classification labels are stratified to preserve class proportions. Training stops with an explanatory error if fewer than 40 usable files or fewer than five records in either class are available.

6. Model implementation

Two scikit-learn pipelines use the same metadata features. `RandomForestRegressor` has 150 trees, maximum depth 8, and minimum leaf size 3. `RandomForestClassifier` uses the same tree settings and balanced class weights. The models are fit only on the training split. Evaluations are calculated only on the held-out test split. Trained pipelines are saved as local Joblib files for inspection or later inference; they should only be loaded from a trusted local source.

7. Evaluation results

Results below come from `analytics/results/metrics.json` after running `train --source generated` with seed 42 and 600 records. They demonstrate the code path on synthetic data. The results must be recomputed with `train --source vault` to make claims about the actual application dataset.

Metric	Result on synthetic demo test set
Training records	450
Test records	150
Large-file threshold	5 MB
Regression R^2	0.5681
Regression MAE (MB)	1.7136
Classification accuracy	0.7800
Classification precision	0.6625
Classification recall	0.8983
Classification F1	0.7626

Metric interpretation: R^2 compares the regressor with predicting the mean size; 1 is perfect and negative values are possible. MAE is the average absolute size error in MB, where lower is better. Precision is the fraction of predicted large files that really are large; recall is the fraction of large files found. F1 combines precision and recall. Accuracy is the overall correct classification rate.

8. Reproduction procedure

From `analytics/`, install `requirements.txt`, set `MONGO_URI` to a MongoDB URI with a database name, and run:

```
python vault_ml.py init-db
python vault_ml.py seed-demo
python vault_ml.py summary --source demo
python vault_ml.py train --source demo
python vault_ml.py train --source vault
```

The last command requires enough real vault records in both size classes. For an offline deterministic check, run `python vault_ml.py train --source generated`. That command writes `results/metrics.json` and two `.joblib` model files without touching MongoDB. Mongo-backed `train` also inserts a document in `ml_model_runs`.

9. Conclusion

The extension demonstrates a complete MongoDB/PyMongo analytics workflow around the existing document vault: indexed operational data, an aggregation pipeline, reproducible demo data, metadata preprocessing, two trained models, and all six requested evaluation metrics. The reported demo metrics describe synthetic file-size patterns only. A real-data study should collect enough consented file metadata, rerun the held-out evaluation, compare simpler baselines, and examine whether MIME-based predictions are useful for the intended product decision. The application also needs separate security and payment-flow remediation before any production use.